AEGISVAULT : A SECURE FILE ENCRYPTION AND DECRYPTION SYSTEM

Project report submitted in partial fulfillment of the requirement for the degree of Bachelor of Technology

In

Computer Science and Engineering

By

DAKSH SHARMA (251034018)

UNDER THE SUPERVISON OF

DR. AMAN SHARMA

Department of Computer Science & Engineering and Information Technology

Jaypee University of Information Technology,Waknaghat, 173234, Himachal Pradesh, INDIA

Acknowledgement

I would like to express my sincere gratitude to my project guide for providing valuable guidance, encouragement, and continuous support throughout the development of this project titled **"AegisVault : A secure file encryption and decryption system."** Their suggestions and technical insights played an important role in the successful completion of my work.

I am also thankful to the faculty members of the Department of Computer Science & Engineering and Information Technology, Jaypee University of Information Technology, for providing the necessary resources and learning environment that enabled me to complete this project successfully.

I would like to extend my heartfelt thanks to my parents and friends for their constant motivation, encouragement, and support during every stage of this project. Their confidence in me inspired me to overcome challenges and remain focused on achieving my goals.

Finally, I would like to express my gratitude to everyone who directly or indirectly contributed to the successful completion of this project.

TABLE OF CONTENTS

Contents

1. Introduction

1.1 Introduction

In today's digital world, the amount of sensitive information stored and shared electronically has increased significantly. Personal documents, financial records, academic files, business data, and confidential information are frequently transferred through digital platforms. This rapid growth has also increased the risk of unauthorized access, data theft, and cyberattacks. Therefore, protecting digital files has become one of the most important aspects of cybersecurity.

File encryption is one of the most effective methods for ensuring data confidentiality. It converts readable information (plaintext) into an unreadable format (ciphertext), making it inaccessible to unauthorized users. Only users possessing the correct decryption key or password can restore the original information. Modern encryption techniques are widely used in banking systems, cloud storage services, secure messaging applications, and enterprise security solutions.

AegisVault is a secure file encryption and decryption application developed using C++ and the **OpenSSL cryptographic library**. The primary objective of this project is to provide a secure, efficient, and user-friendly solution for protecting files against unauthorized access. The application supports password-based encryption, allowing users to encrypt files into a custom `.aegis` format and later restore them using the correct password.

The application implements the **Advanced Encryption Standard (AES-256)** in **Cipher Block Chaining (CBC)** mode to ensure strong data confidentiality. Instead of directly using the user password as an encryption key, the project generates a secure **256-bit encryption key** using the **Password-Based Key Derivation Function 2 (PBKDF2)** with the **SHA-256** hashing algorithm. A randomly generated **16-byte salt** strengthens password security by preventing dictionary and rainbow table attacks, while a randomly generated **16-byte Initialization Vector (IV)** ensures that encrypting the same file with the same password produces different ciphertext each time.

AegisVault also introduces a **custom encrypted file format** that stores important metadata such as the file header, version number, original file extension, cryptographic salt, and initialization vector before the encrypted data. This design enables the application to accurately reconstruct the original file after successful decryption while maintaining compatibility with future versions of the software.

The developed application supports both **text and binary files**, including documents, PDF files, images, compressed archives, and other commonly used file formats. During testing, successful encryption and decryption were performed on multiple file types while preserving the integrity and usability of the original data.

Overall, **AegisVault** demonstrates the practical implementation of modern cryptographic principles using industry-standard libraries and secure programming practices. The project provides valuable experience in cryptography, secure file handling, password-based authentication, and software development using C++, making it an effective educational and cybersecurity-focused application.

1.2 Objectives

The primary objective of the **AegisVault** project is to develop a secure and efficient file encryption and decryption application that protects sensitive information from unauthorized access using modern cryptographic techniques. The project focuses on implementing industry-standard encryption methods while providing a simple and user-friendly interface for file protection.

The specific objectives of this project are:

- To develop a secure file encryption and decryption application using the C++ programming language.
- To implement the **AES-256-CBC** encryption algorithm using the OpenSSL cryptographic library.
- To generate a secure 256-bit encryption key from a user-defined password using the **PBKDF2 key derivation algorithm** with **SHA-256 hashing**.
- To enhance password security by using a randomly generated cryptographic salt for every encryption process.
- To generate a unique Initialization Vector (IV) for each encryption operation to ensure different ciphertext for identical files encrypted with the same password.
- To design and implement a **custom .aegis** encrypted file format capable of storing metadata such as file version, original file extension, salt, and initialization vector.
- To support the encryption and decryption of both text and binary files, including documents, images, PDF files, and compressed archives.
- To validate the correctness of the decryption process by allowing only users with the correct password to recover the original file.
- To provide appropriate error handling for incorrect passwords and invalid encrypted files.
- To demonstrate the practical application of cryptography and secure programming concepts using industry-standard libraries.

1.3 Motivation

The motivation behind developing **AegisVault** originated from the growing importance of data security in today's digital environment. As individuals and organizations increasingly rely on digital storage for personal, academic, and professional information, protecting confidential files from unauthorized access has become a critical requirement. Cyber threats such as data breaches, ransomware attacks, unauthorized file access, and password theft continue to increase, making file encryption an essential security measure rather than an optional feature.

During the study of cybersecurity and cryptography, it was observed that many users understand the importance of protecting sensitive information but lack practical knowledge of how encryption systems function internally. Most existing encryption tools operate as black-box applications, allowing users to encrypt files without understanding the underlying cryptographic processes. This project was therefore undertaken to bridge the gap between theoretical cryptography concepts and their practical implementation.

Another important motivation was to gain hands-on experience with industry-standard cryptographic libraries and secure programming practices. Instead of implementing encryption algorithms from scratch, the project utilizes the **OpenSSL** library, which is widely adopted in professional software applications. This approach provides practical exposure to real-world cryptographic APIs while following secure software development practices.

The project also serves as an opportunity to strengthen programming skills in **C++**, particularly in areas such as file handling, memory management, modular programming, binary file processing, and integration of external libraries. Developing a complete encryption and decryption application allowed the practical application of concepts learned during coursework while enhancing problem-solving and software development abilities.

Furthermore, **AegisVault** was developed with the objective of creating a simple, secure, and educational application that demonstrates the complete workflow of password-based file encryption. The project not only fulfills academic requirements but also serves as a strong foundation for future enhancements, including authenticated encryption, secure vault management, and advanced cybersecurity features.

1.4 Project Methodology

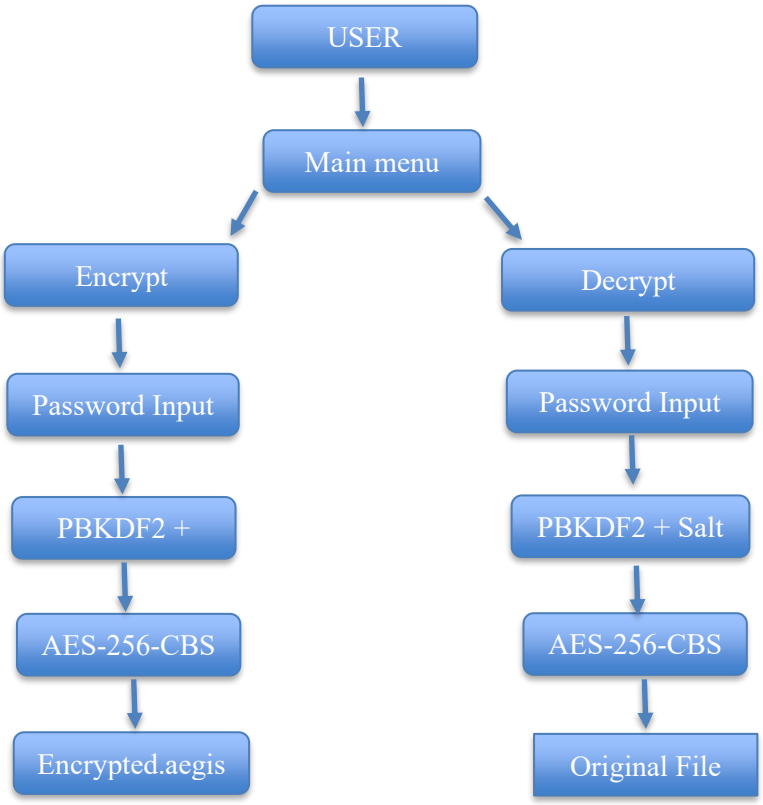
1.4.1 System Overview

The **AegisVault** system follows a modular architecture in which each module performs a specific task during the encryption and decryption process. The application is developed using **C++** and the **OpenSSL** cryptographic library, allowing secure implementation of modern cryptographic algorithms while maintaining modularity and code readability.

The workflow begins when the user selects either the encryption or decryption operation from the main menu. During encryption, the user provides the input file and a password. A random cryptographic salt is generated and used together with the password to derive a secure **256-bit encryption key** through **the PBKDF2** key derivation algorithm. Simultaneously, a random 16-byte **Initialization Vector (IV)** is generated to ensure uniqueness of the encryption process.

The derived key and IV are then supplied to the **AES-256-CBC** encryption algorithm implemented using the OpenSSL EVP interface. After successful encryption, the application stores the encrypted data inside a **custom.aegis** file along with metadata including the file header, version number, original file extension, salt, and IV.

During decryption, the application reads the metadata from the encrypted file, derives the same encryption key from the user-provided password using the stored salt, and decrypts the ciphertext using the stored IV. If the password is correct, the original file is successfully restored with its original extension. If an incorrect password or an invalid encrypted file is detected, the application terminates the operation and displays an appropriate error message.



1.4.2 Encryption Process

The encryption process in **AegisVault** begins when the user selects the Encrypt File option from the main menu. The application prompts the user to enter the name of the input file along with a password. The file is read in binary mode to ensure that both text and binary file formats can be processed without data loss.

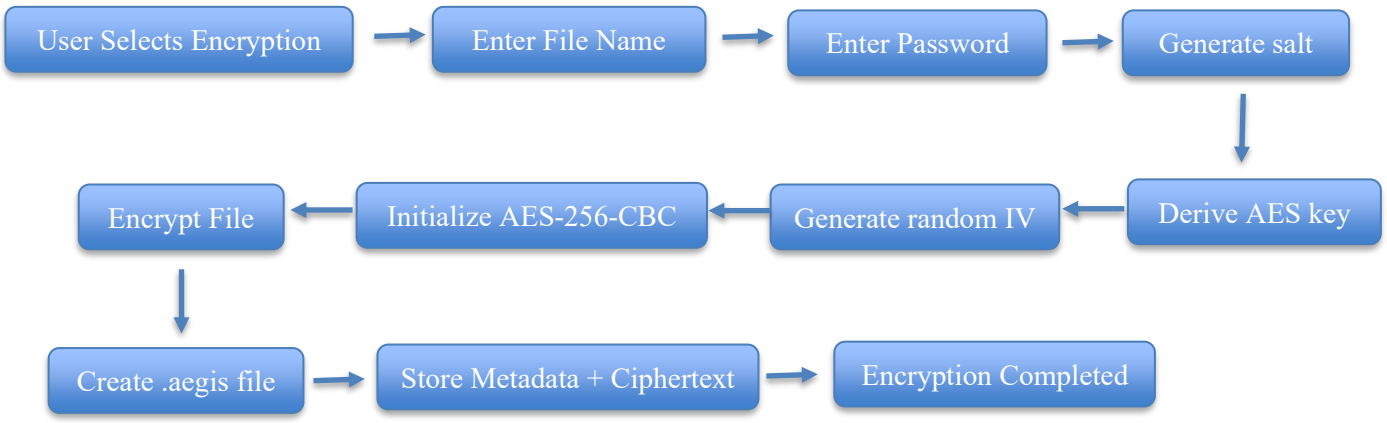
After receiving the password, the application generates a 16-byte random salt using the **OpenSSL** cryptographic random number generator. The password and salt are then supplied to the **PBKDF2 (Password-Based Key Derivation Function 2)** algorithm with the **SHA-256** hashing function. **PBKDF2** performs **100,000 iterations** to derive a secure 256-bit encryption key, making password-based attacks significantly more difficult.

Next, the application generates a random 16-byte **Initialization Vector (IV)**. The IV ensures that even if the same file is encrypted multiple times using the same password, the resulting ciphertext remains different, thereby improving security.

The application then initializes the **AES-256-CBC** encryption algorithm through the OpenSSL EVP interface. The plaintext data is encrypted block by block, producing ciphertext that cannot be interpreted without the correct encryption key.

Once encryption is completed, the application creates a **custom.aegis** file. Before storing the ciphertext, it writes important metadata including the file header, version number, original file extension, generated salt, and initialization vector. Finally, the encrypted data appended to the file, and the encryption context is released from memory.

The entire process ensures that sensitive files remain confidential and can only be restored by users possessing the correct password.



```
PS C:\Users\DELL\OneDrive\Desktop\CYBER SECURITY\AegisVault> .\build.ps1

Build Successful!

------------------------------------------------------------
                AEGISVAULT encryption system
------------------------------------------------------------

1. Encrypt files
2. Decrypt files
3. To Exit

Enter your choice : 1
------------------------------------------------------------
                 ENCRYPTION MODULE STARTED!
------------------------------------------------------------

Enter your file name with extension : sample.txt
Enter your password : sample
Reading file...

Generated Salt :
5c85e77e4c1ace93ffc9815063e33718

Generated IV(16 bytes):
ae71f8c683a29499cc1ce7861c4ce391

File encrypted successfully!
```

Figure 1 : Successful File Encryption Using AegisVault

1.4.3 Decryption Process

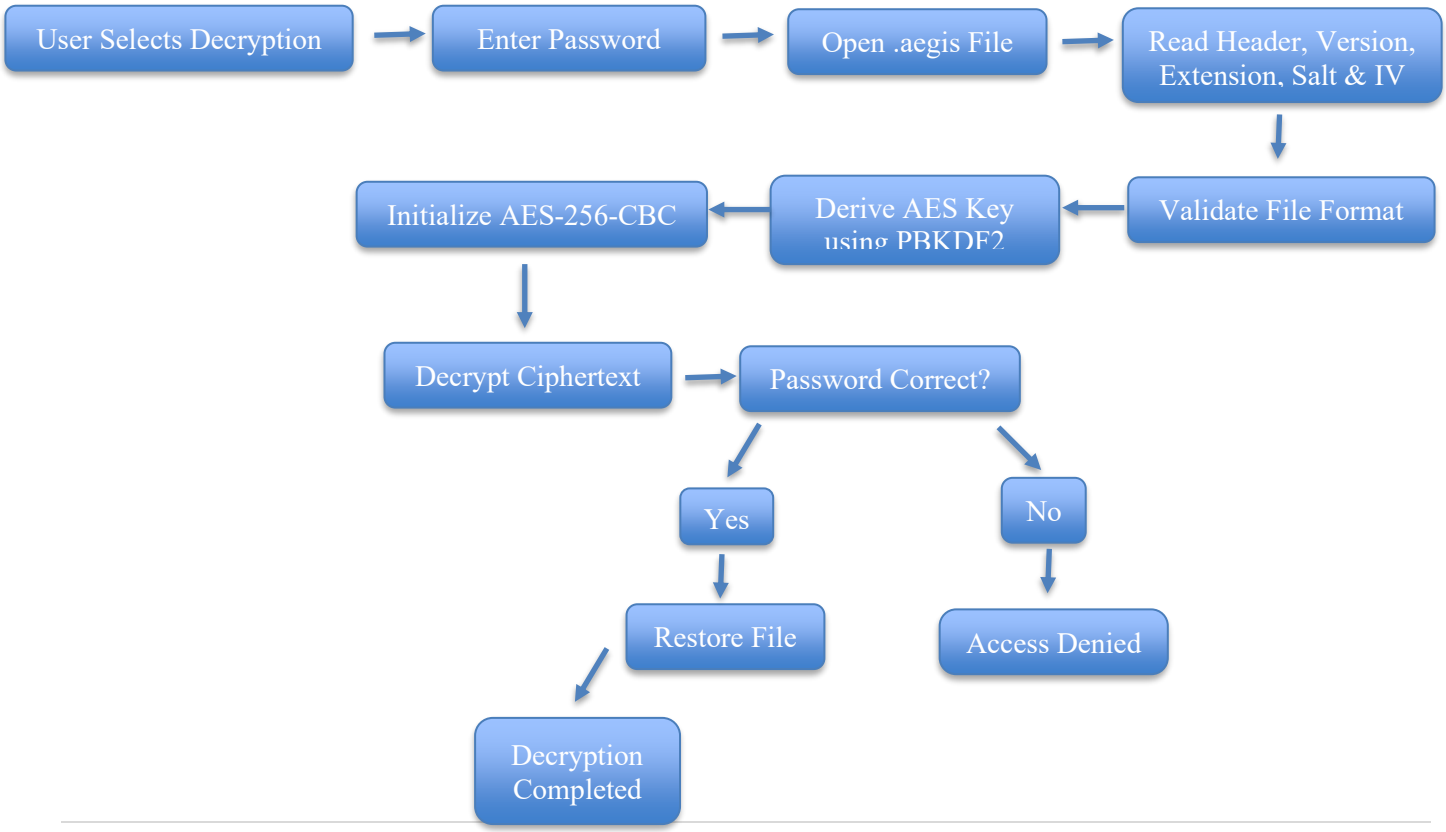
The decryption process begins when the user selects the **Decrypt File** option from the main menu. The application prompts the user to enter the password associated with the encrypted file. It then opens the **encrypted.aegis** file in binary mode and reads the stored metadata, including the file header, version number, original file extension, cryptographic salt, and **initialization vector (IV).**

Before starting the decryption process, the application verifies the file header and version number to ensure that the selected file is a valid **AegisVault** encrypted file. If the header or version does not match the expected values, the application terminates the operation and displays an appropriate error message.

Using the password entered by the user and the stored salt, the application derives the same **256-bit AES encryption key** through the **PBKDF2** key derivation algorithm with SHA-256. Since the same password, salt, and iteration count are used, the generated key matches the one created during encryption.

The stored initialization vector is then supplied to the **AES-256-CBC** decryption algorithm through the **OpenSSL** EVP interface. The ciphertext is decrypted back into its original plaintext form. If the password is incorrect or the encrypted data has been modified, the decryption process fails during the final verification stage, and the application displays an **Access Denied** message without generating the output file.

If decryption is successful, the application restores the original file using the stored file extension and saves it in the output directory. Finally, the **OpenSSL** decryption context is released, completing the decryption process securely.



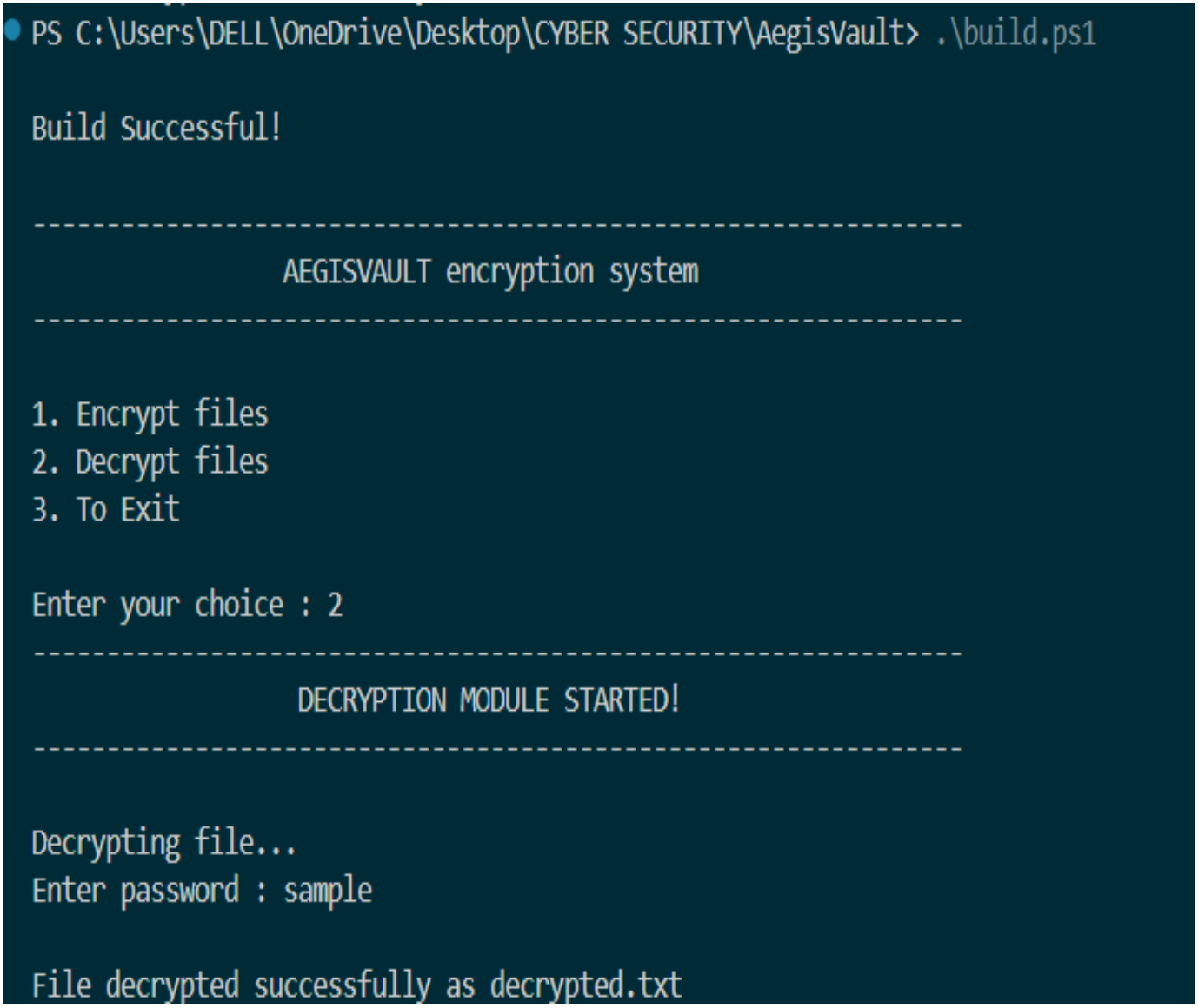


Figure 2 : Successful File Decryption Using AegisVault

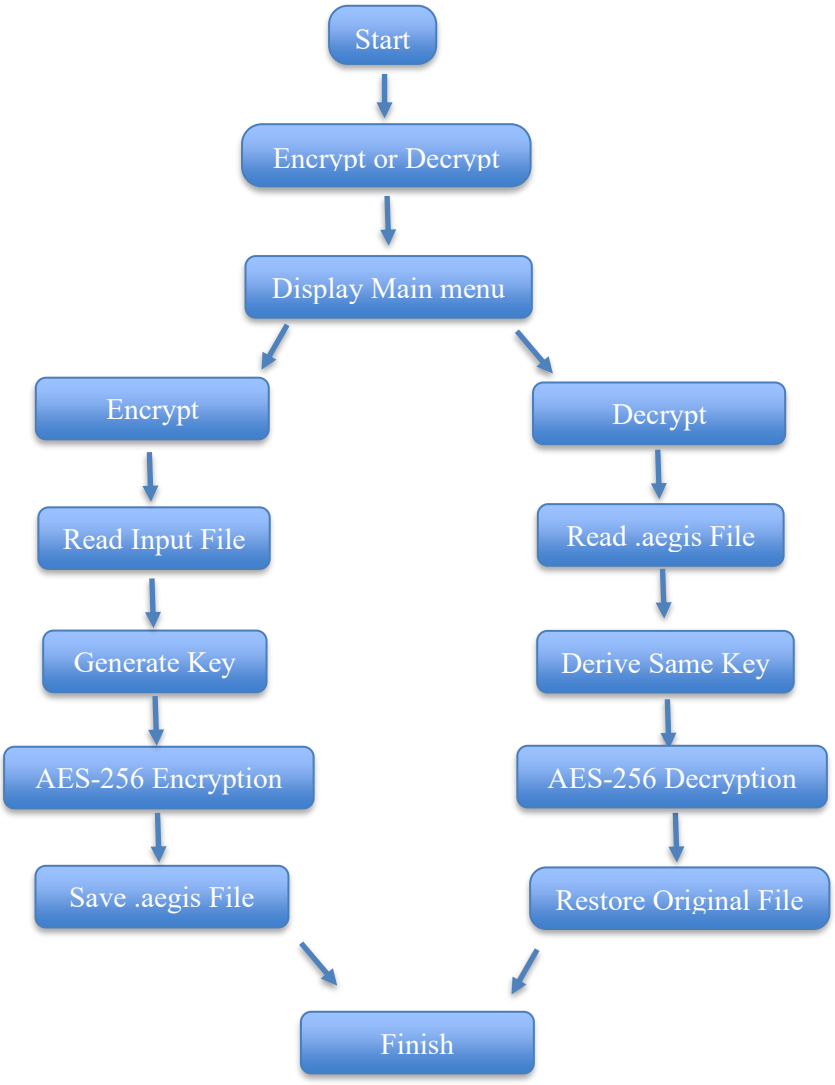
1.4.4 Project Workflow

The overall workflow of **AegisVault** is designed to provide secure and reliable file encryption and decryption through a sequence of well-defined operations. The application begins by displaying a menu that allows the user to choose between encryption and decryption.

If the user selects the encryption option, the application accepts the input file and password, generates a random cryptographic salt and initialization vector, derives a secure **AES-256** encryption key using **PBKDF2** with **SHA-256**, encrypts the file using the **AES-256-CBC** algorithm, and stores the encrypted data together with the required metadata in a **custom.aegis** file.

If the user selects the decryption option, the application reads the encrypted file, verifies the file header and version, extracts the stored metadata, derives the encryption key using the provided password and stored salt, and decrypts the ciphertext using the stored initialization vector. If the password is correct, the original file is restored successfully; otherwise, the application displays an appropriate error message and terminates the operation.

This workflow ensures confidentiality, maintains data integrity during encryption and decryption, and provides secure password-based access to protected files.



1.5 Outcomes

The successful completion of the **AegisVault** project resulted in the development of a secure, modular, and user-friendly file encryption and decryption application based on modern cryptographic standards. The project demonstrates the practical implementation of secure software development principles using **C++** and the **OpenSSL** cryptographic library.

The major deliverables and outcomes of the project are as follows:

- A secure password-based file encryption and decryption application developed using **C++.**
- Implementation of the **AES-256-CBC** encryption algorithm through the **OpenSSL EVP API.**
- Secure key generation using **PBKDF2** with **SHA-256** and 100,000 iterations.
- Random generation of a 16-byte cryptographic salt and a 16-byte Initialization Vector (IV) for every encryption process.
- Design and implementation of a **custom .aegis** encrypted file format capable of storing encryption metadata.
- Support for encryption and decryption of both text and binary files, including PDF documents, images, compressed archives, and text files.
- Automatic restoration of the original file extension after successful decryption.
- Validation mechanisms for detecting invalid encrypted files and incorrect passwords.
- A modular source code structure separating encryption, decryption, cryptographic utilities, key derivation, and user interface into independent modules.
- Successful testing of the application on multiple file formats to verify correctness, reliability, and data integrity.

The project provides practical exposure to cryptographic programming, secure file handling, password-based authentication, and modular software development. It also establishes a strong foundation for future enhancements such as authenticated encryption, digital signatures, graphical user interfaces, cloud-based secure storage, and multi-user access control.

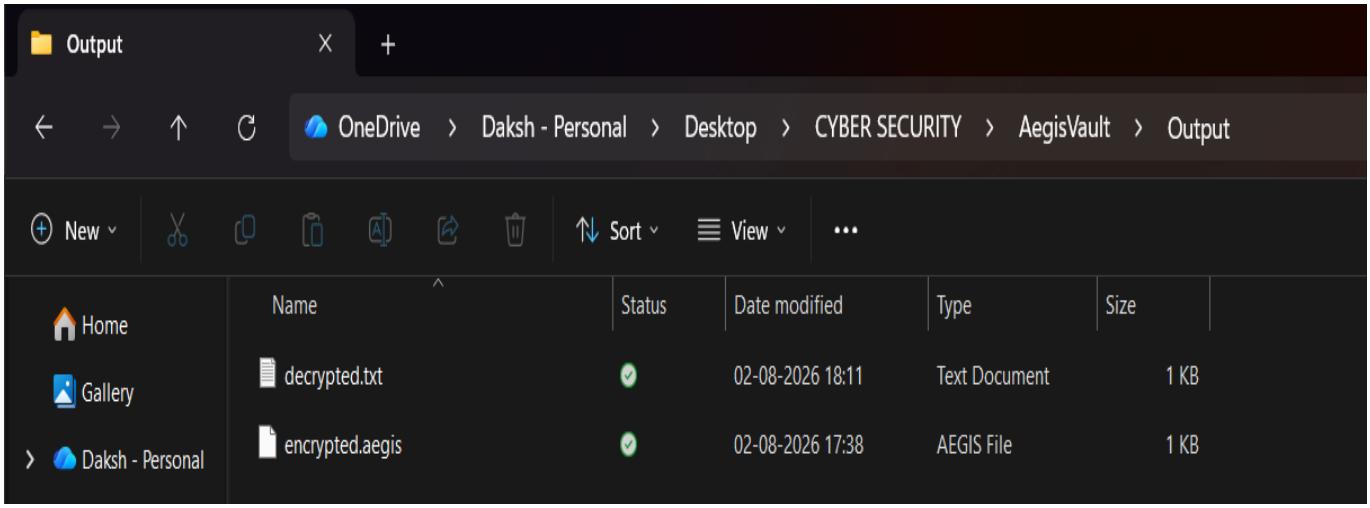


Figure 3 : Final Output Generated by AegisVault

2. Technologies Used

2.1 Programming Language (C++)

C++ is the primary programming language used for developing the **AegisVault** application. It is a high-performance, general-purpose programming language that provides direct access to system resources and efficient memory management, making it well suited for security-focused applications.

The project uses **C++** to implement the complete encryption and decryption workflow, including user interaction, file handling, modular programming, binary data processing, and integration with external cryptographic libraries. Features such as functions, header files, vectors, strings, and file streams have been extensively utilized to maintain modularity and improve code readability.

The choice of **C++** also enables seamless integration with the **OpenSSL** library, allowing secure implementation of cryptographic algorithms while maintaining high execution efficiency.

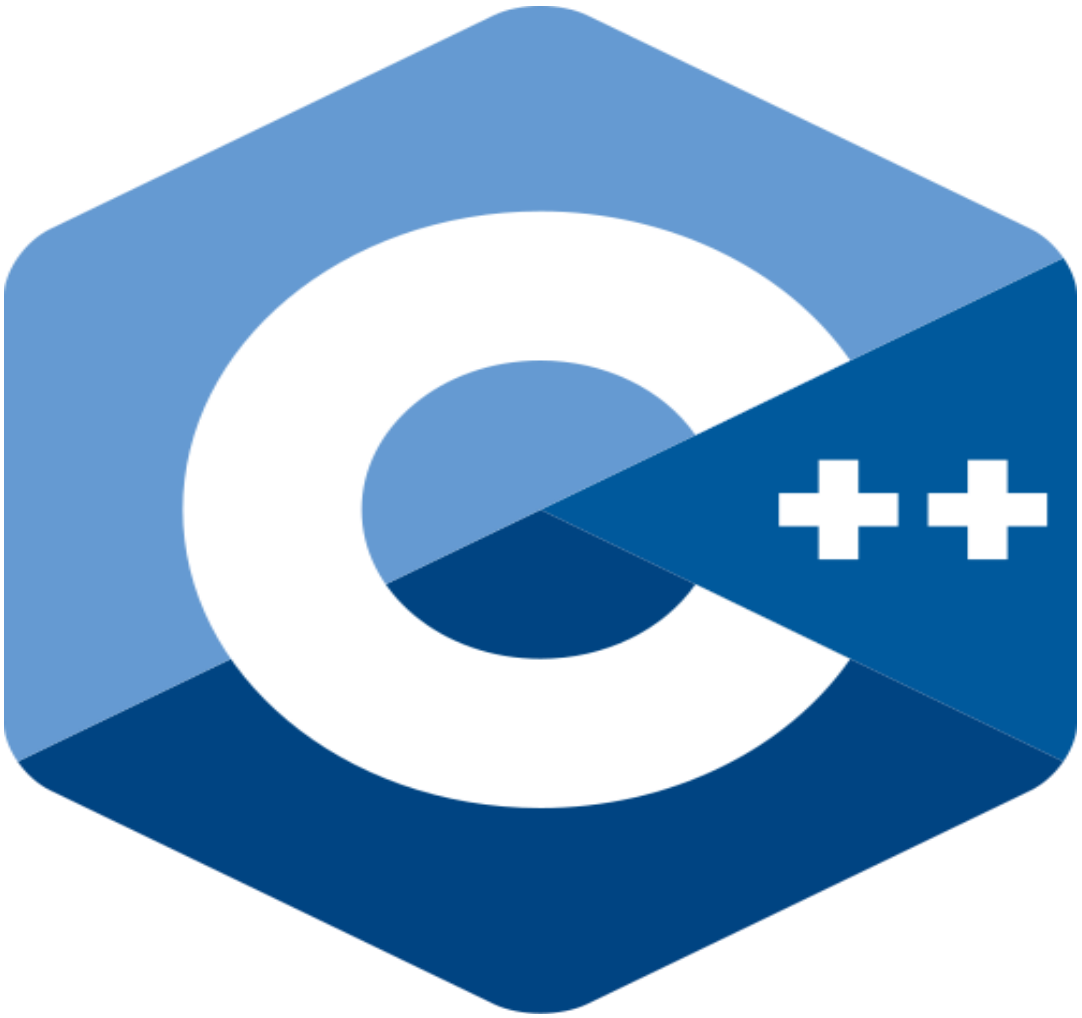


Figure 4 : C++ Programming Language

2.2 Cryptographic Library (OpenSSL)

OpenSSL is an open-source cryptographic library used in the **AegisVault** project to implement secure encryption and decryption functionalities. It provides a wide range of industry-standard cryptographic algorithms and security protocols, making it one of the most trusted libraries for developing secure applications.

In this project, **OpenSSL** is used through the **EVP (Envelope)** interface, which provides a high-level and flexible API for performing cryptographic operations. The library is responsible for initializing the **AES-256-CBC** encryption algorithm, encrypting plaintext into ciphertext, decrypting ciphertext back into its original form, generating cryptographically secure random values for the salt and **Initialization Vector (IV)**, and deriving secure encryption keys through **PBKDF2**.

Using OpenSSL eliminates the need to implement cryptographic algorithms manually, reducing the possibility of security vulnerabilities and ensuring compliance with established cryptographic standards. Its optimized implementation also provides efficient performance while maintaining a high level of security.

The integration of **OpenSSL** with **C++** allows **AegisVault** to securely protect user files using reliable and well-tested cryptographic techniques.

Figure 5 : OpenSSL Cryptographic Library

2.3 Encryption Algorithm (AES-256-CBC)

The **AegisVault** project uses the **Advanced Encryption Standard (AES)** with a **256-bit key** operating in **Cipher Block Chaining (CBC)** mode to provide strong and reliable file encryption. AES is a symmetric-key encryption algorithm, meaning that the same cryptographic key is used for both encryption and decryption.

AES-256 is widely recognized as one of the most secure encryption standards and is adopted by governments, financial institutions, cloud service providers, and cybersecurity organizations worldwide. The **256-bit** key length offers an extremely large key space, making brute-force attacks computationally impractical with current technology.

In **CBC mode**, each plaintext block is combined with the previous ciphertext block before encryption. For the first block, a randomly generated **Initialization Vector (IV)** is used. This approach ensures that identical plaintext blocks do not produce identical ciphertext, thereby improving confidentiality and preventing pattern analysis.

Within **AegisVault, AES-256-CBC** is implemented using the **OpenSSL EVP interface**, which provides a secure and standardized method for performing encryption and decryption. The application initializes the encryption context, processes the file data block by block, and securely finalizes the encryption operation before storing the encrypted output in the **custom.aegis** file format.

The use of **AES-256-CBC** ensures that sensitive user files remain protected while maintaining compatibility with widely accepted cryptographic standards.

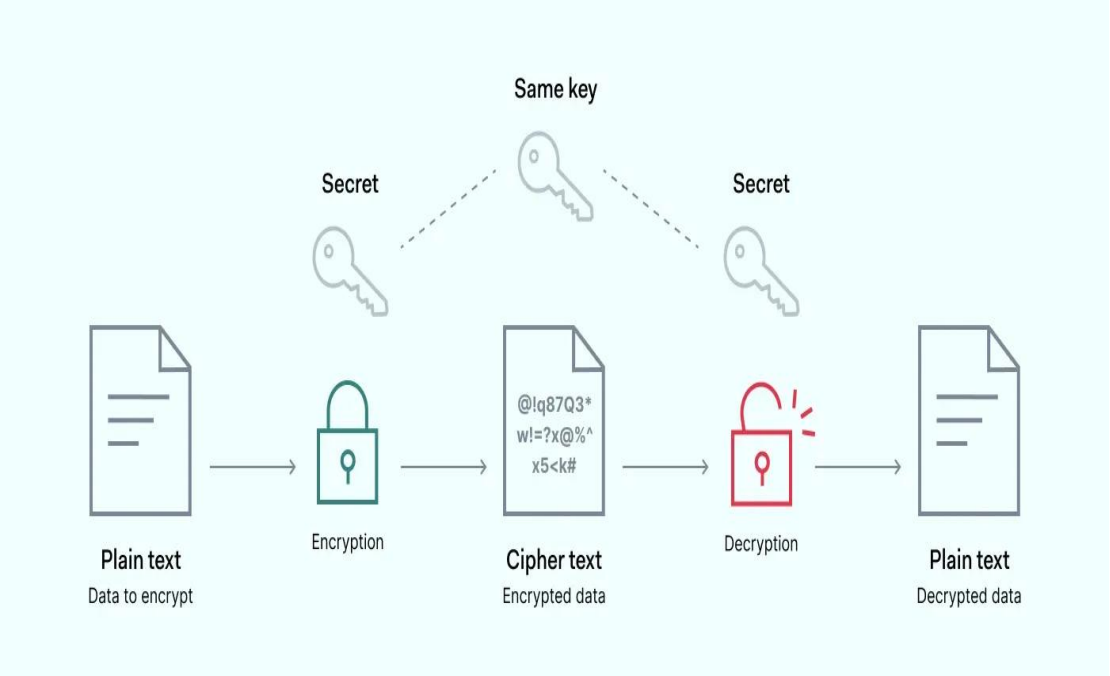


Figure 6 : AES-256-CBC Encryption Algorithm

2.4 Key Derivation Function (PBKDF2)

The **Password-Based Key Derivation Function 2 (PBKDF2)** is a cryptographic key derivation algorithm used to generate a secure encryption key from a user-provided password. Instead of using the password directly as the encryption key**, PBKDF2** transforms it into a strong 256-bit cryptographic key through multiple iterations of a secure hash function.

In the **AegisVault** project, PBKDF2 is implemented using the **SHA-256** hashing algorithm with **100,000 iterations**. During encryption, the application generates a random **16-byte cryptographic salt**, which is combined with the user password before the key derivation process begins. The resulting 256-bit key is then supplied to the **AES-256-CBC** encryption algorithm.

The use of **PBKDF2** significantly improves password security. The randomly generated salt prevents attackers from using precomputed rainbow tables, while the large number of iterations increases the computational effort required for brute-force and dictionary attacks. Even if two users choose the same password, different salts produce different encryption keys, enhancing overall security.

During decryption, the application reads the stored salt from the **encrypted.aegis** file and performs the same **PBKDF2** operation using the user-provided password. If the password is correct, the generated key matches the original encryption key, allowing successful decryption. Otherwise, the decryption process fails, protecting the encrypted data from unauthorized access.

The integration of **PBKDF2** ensures that **AegisVault** follows secure password-based encryption practices while complying with modern cryptographic recommendations.

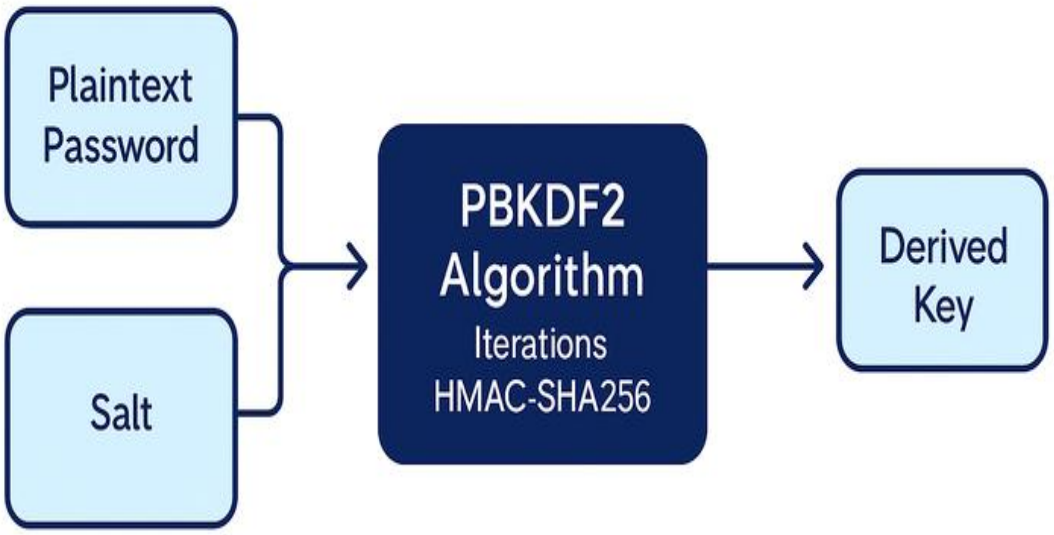


Figure 7 : PBKDF2 Key Derivation Process

2.5 Development Environment

The **AegisVault** project was developed and tested in a Windows-based development environment using modern programming tools and libraries. The development environment was carefully selected to ensure compatibility with the C++ programming language and the **OpenSSL** cryptographic library.

The source code was written and managed using **Visual Studio Code (VS Code)**, which provided features such as syntax highlighting, code completion, debugging support, and integrated terminal access. VS Code simplified project organization by allowing the source files to be managed through separate modules for encryption, decryption, cryptographic utilities, and key derivation.

The project was compiled using the **MSYS2 MinGW-w64 GCC Compiler**, which provides a modern GNU Compiler Collection (GCC) implementation for Windows. This compiler supports the C++ language standard required by the project and allows seamless integration with external libraries such as OpenSSL.

The cryptographic functions were implemented using the **OpenSSL** library, which was linked with the project during compilation. OpenSSL provided secure implementations of AES-256-CBC encryption, PBKDF2 key derivation, SHA-256 hashing, random salt generation, and initialization vector generation.

The application was developed and tested on **Windows 11**, ensuring compatibility with the operating system while providing a stable platform for file handling and cryptographic operations.

The combination of Visual Studio Code, MSYS2 MinGW-w64, OpenSSL, and Windows 11 created a reliable and efficient development environment for building and testing the **AegisVault** application.

2.6Custom File Format (.aegis)

AegisVault uses a custom encrypted file format with the `.aegis` extension to securely store encrypted data along with the metadata required for successful decryption. Instead of saving only the ciphertext, the application embeds additional information that enables the decryption module to reconstruct the original file accurately.

The .aegis file begins with a unique **file header** that identifies the file as an AegisVault encrypted file. A version number is stored immediately after the header to ensure compatibility with future versions of the application. The original file extension is also stored so that the decrypted file can be restored with its original format.

Following the metadata, the application stores the **16-byte cryptographic salt** and the **16-byte Initialization Vector (IV)** generated during encryption. These values are essential for deriving the correct encryption key through PBKDF2 and for performing AES-256-CBC decryption.

Finally, the encrypted ciphertext is written to the file. During decryption, the application reads the metadata in the same order, verifies the file header and version, derives the encryption key using the stored salt and user password, and decrypts the ciphertext using the stored IV.

The custom.aegis format provides a structured and secure method of storing encrypted information while allowing future extensions without affecting backward compatibility.

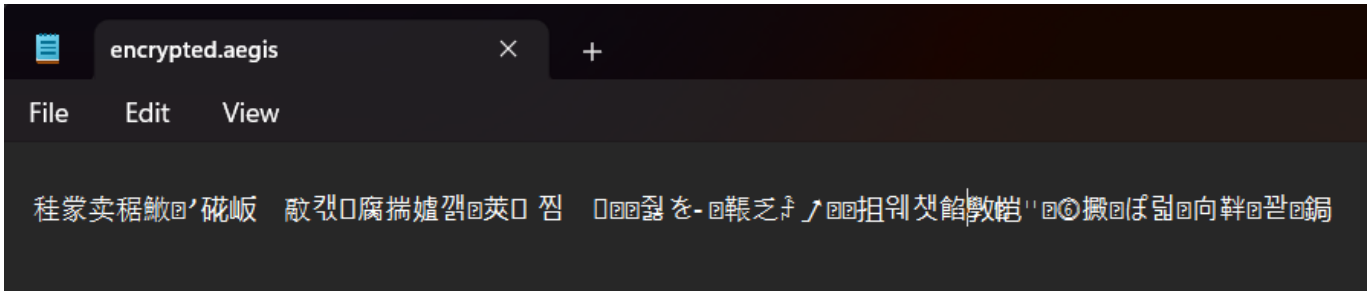
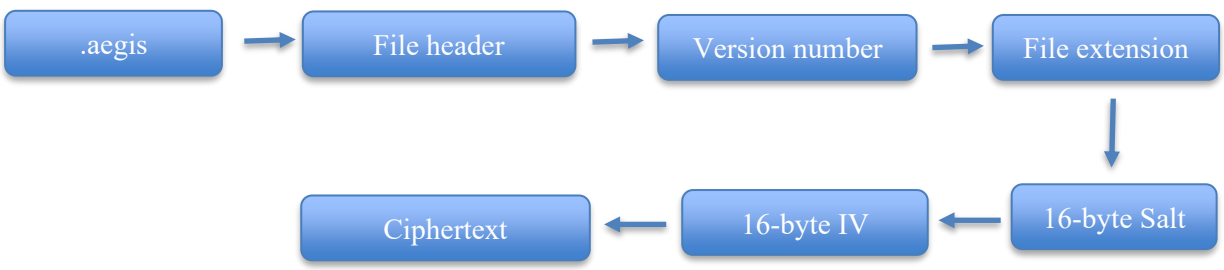


Figure 8 : Generated AegisVault Encrypted File



3. System Design

3.1 Overall System Architecture

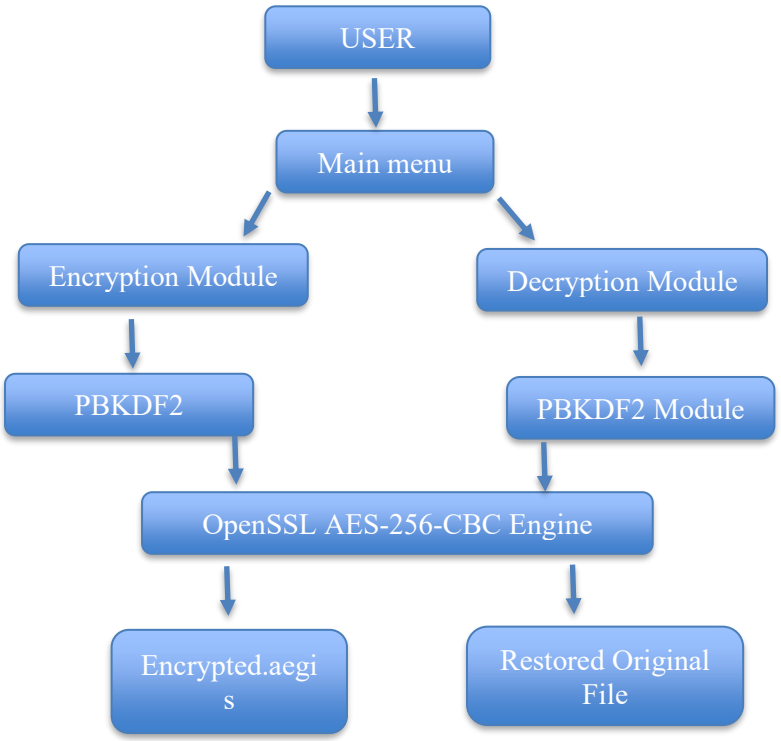
The AegisVault system follows a modular architecture in which each component performs a specific responsibility during the encryption and decryption process. The architecture is designed to improve maintainability, code readability, and scalability by separating different functionalities into independent modules.

The application begins with the **Main Menu Module**, which allows the user to select either file encryption or file decryption. Based on the user's selection, control is transferred to the corresponding module.

During encryption, the **Encryption Module** accepts the input file and user password. The password is securely transformed into a 256-bit encryption key through the **PBKDF2 Module**, while the **Cryptographic Utility Module** generates a random salt and Initialization Vector (IV). These cryptographic values are then supplied to the AES-256-CBC algorithm implemented through the OpenSSL library to encrypt the file. Finally, the encrypted data and required metadata are stored inside the custom .aegis file.

During decryption, the **Decryption Module** reads the encrypted .aegis file, verifies the file header and version, extracts the stored metadata, derives the encryption key using the PBKDF2 module, and decrypts the ciphertext using AES-256-CBC. If the supplied password is valid, the original file is restored successfully; otherwise, the application terminates the process with an appropriate error message.

This modular architecture improves maintainability by allowing each component to perform a dedicated function while minimizing dependencies between modules.



3.2 Module Description

The AegisVault application follows a modular programming approach, where each source file is responsible for a specific functionality. This modular design improves code readability, simplifies debugging, and allows individual components to be maintained or upgraded without affecting the overall application.

The project is divided into the following modules:

1. Main Module (main.cpp)
The main module serves as the entry point of the application. It displays the main menu, accepts the user's choice, and transfers program control to either the encryption or decryption module. This module is responsible for managing the overall program flow.

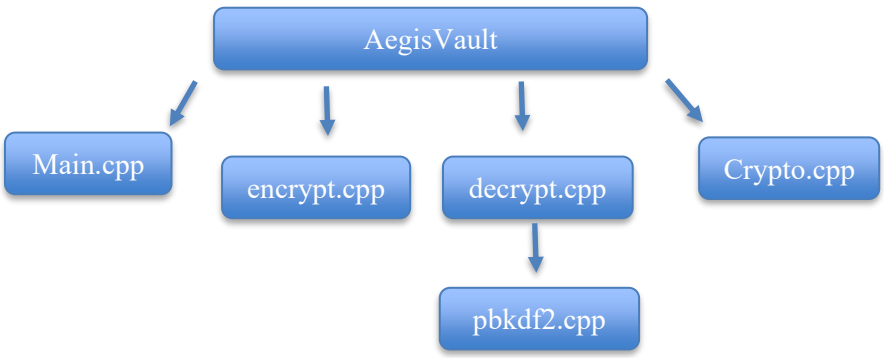
2. Encryption Module (encrypt.cpp)
The encryption module performs the complete encryption process. It reads the input file, accepts the user's password, generates the cryptographic salt and Initialization Vector (IV), derives the AES encryption key using PBKDF2, encrypts the file using AES-256-CBC through OpenSSL, and stores the encrypted data in the custom .aegis file format.

3. Decryption Module (decrypt.cpp)
The decryption module restores encrypted files. It reads the metadata from the .aegis file, verifies the file format, derives the encryption key using the supplied password and stored salt, decrypts the ciphertext using AES-256-CBC, and restores the original file with its original extension. It also detects incorrect passwords and invalid encrypted files.

4. Cryptographic Utility Module (crypto.cpp)
This module provides cryptographic utility functions required by both the encryption and decryption processes. It securely generates random salts and Initialization Vectors (IVs) using the OpenSSL random number generator.

5. Key Derivation Module (pbkdf2.cpp)
This module implements the Password-Based Key Derivation Function (PBKDF2). It derives a secure 256-bit AES key from the user's password using SHA-256, a random salt, and 100,000 iterations.

6. Header Files
The project uses separate header files (encrypt.h, decrypt.h, crypto.h, pbkdf2.h, and menu.h) to declare functions and share interfaces between different modules. This improves modularity and reduces code duplication.



3.3 Encryption Module Design

The Encryption Module is responsible for securely converting the original file into an encrypted file using password-based encryption. The module is implemented in encrypt.cpp and integrates cryptographic functions provided by the OpenSSL library.

The encryption process begins by accepting the input file name and the user-defined password. The selected file is opened in binary mode and its contents are read into memory to ensure that both text and binary files can be processed correctly.

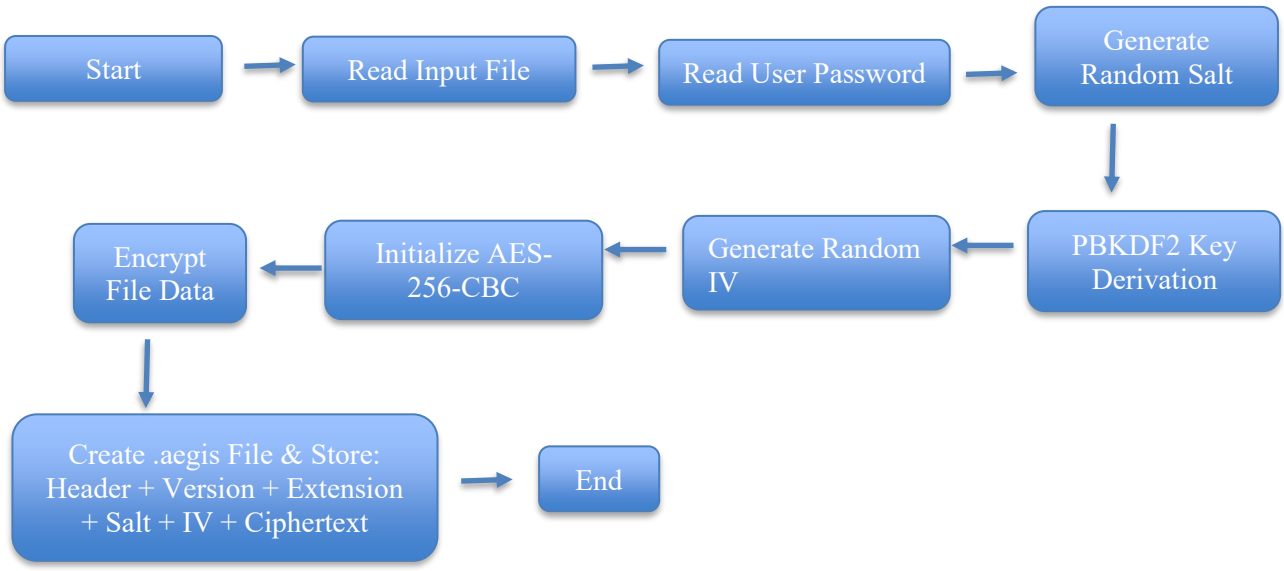
A random 16-byte cryptographic salt is then generated using the OpenSSL random number generator. The user password and the generated salt are supplied to the PBKDF2 key derivation algorithm, which performs 100,000 SHA-256 iterations to generate a secure 256-bit AES encryption key.

After the encryption key has been generated, the application creates a random 16-byte Initialization Vector (IV). The IV ensures that encrypting the same file with the same password produces different ciphertext each time.

The module initializes the AES-256-CBC encryption algorithm through the OpenSSL EVP interface. The plaintext file is encrypted block by block until the entire file has been processed.

Once encryption is complete, the application creates a custom .aegis file. Before writing the encrypted data, it stores the file header, version number, original file extension, generated salt, and Initialization Vector. Finally, the ciphertext is appended to the file, producing the final encrypted output.

This design ensures confidentiality, protects user passwords through secure key derivation, and preserves all information required for successful decryption.



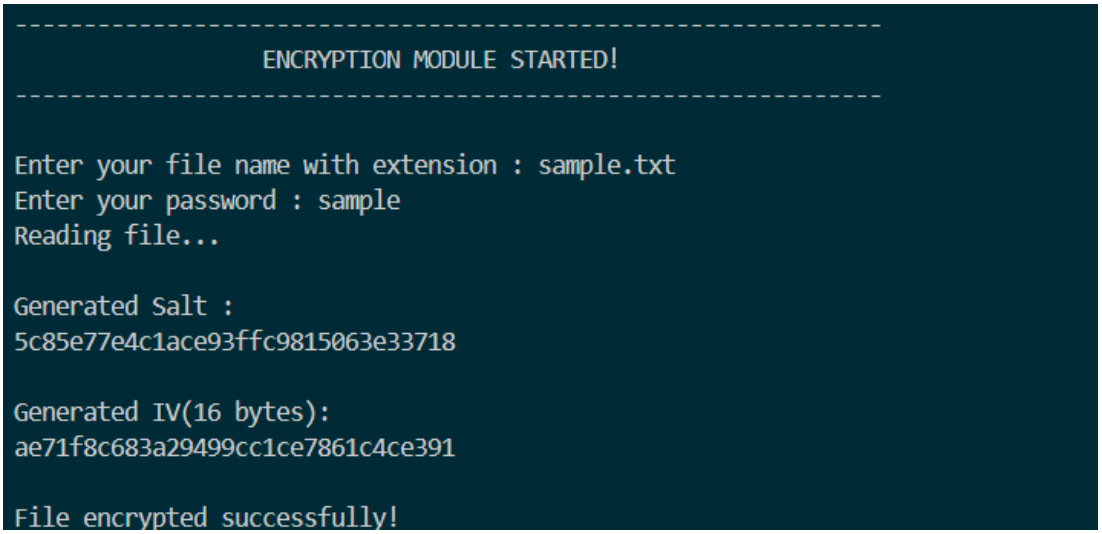


Figure 9: Execution of the Encryption Module

3.4 Decryption Module Design

The Decryption Module is responsible for restoring the original file from the encrypted .aegis file. This module is implemented in decrypt.cpp and uses the same cryptographic parameters that were generated during the encryption process.

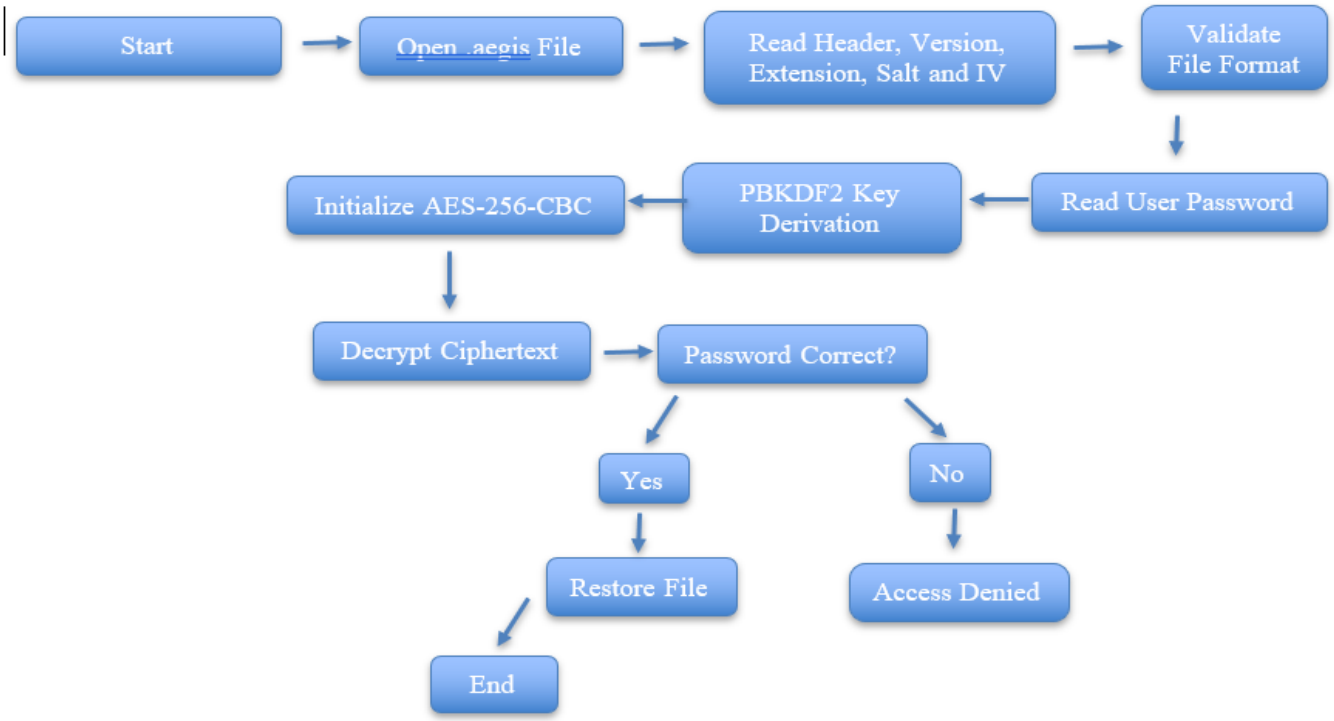
The decryption process begins when the user selects the decryption option from the main menu and enters the password. The application opens the encrypted .aegis file in binary mode and reads the stored metadata, including the file header, version number, original file extension, cryptographic salt, and Initialization Vector (IV).

Before proceeding with decryption, the module validates the file header and version number to ensure that the selected file was created by AegisVault. If the validation fails, the application terminates the process and displays an appropriate error message.

The stored salt and the user-provided password are supplied to the PBKDF2 key derivation function to regenerate the same 256-bit AES encryption key that was created during encryption. The stored IV is then used to initialize the AES-256-CBC decryption algorithm through the OpenSSL EVP interface.

The ciphertext is decrypted block by block until the complete plaintext is recovered. If the entered password is incorrect or the encrypted file has been modified, the final verification step fails, and the application displays an Access Denied message without creating the output file.

When the password is correct, the original file is restored using the stored file extension and saved in the output directory. Finally, the cryptographic context is released, completing the decryption process securely.



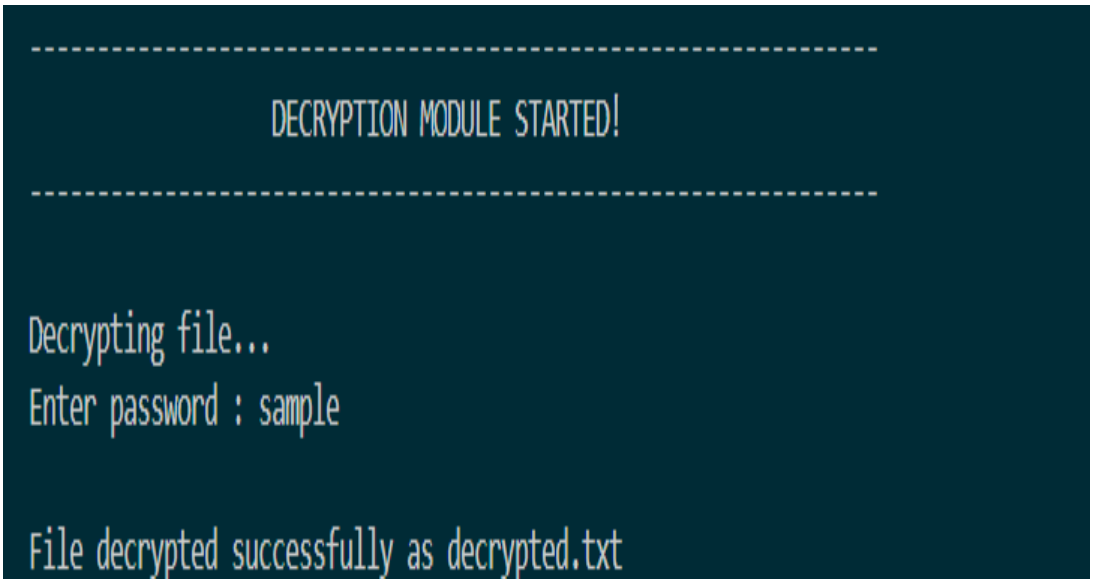


Figure 10: Execution of the Decryption Module

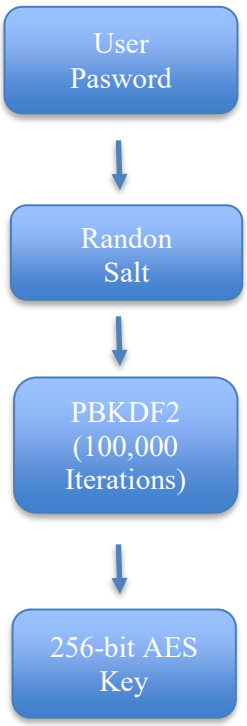
3.5 Key Derivation Module (PBKDF2)

The Key Derivation Module is implemented in pbkdf2.cpp and is responsible for generating a secure 256-bit encryption key from the user-provided password. Instead of using the password directly, the module employs the Password-Based Key Derivation Function 2 (PBKDF2) with the SHA-256 hashing algorithm.

A randomly generated 16-byte salt is combined with the user's password before the key derivation process begins. The PBKDF2 algorithm performs **100,000 iterations**, significantly increasing the computational effort required for brute-force attacks while producing a strong encryption key suitable for AES-256.

The same password, salt, and iteration count are used during decryption to regenerate the identical encryption key. This ensures that only users who provide the correct password can successfully decrypt the protected file.

The module plays a critical role in improving password security and protecting encrypted files against common password-based attacks.



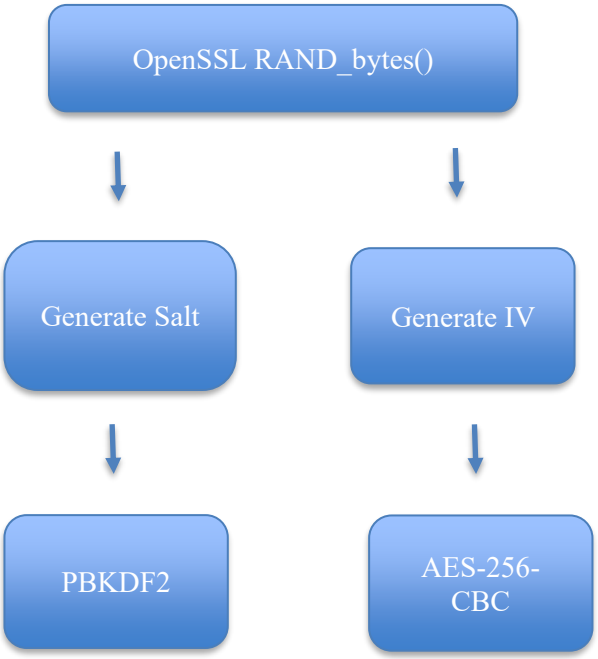
3.6 Cryptographic Utility Module

The Cryptographic Utility Module is implemented in crypto.cpp and provides the supporting cryptographic functions required by both the encryption and decryption modules. Its primary responsibility is to generate secure random values that strengthen the encryption process.

The module generates a **16-byte cryptographic salt** using OpenSSL's secure random number generator. This salt is supplied to the PBKDF2 algorithm during key derivation, ensuring that identical passwords produce different encryption keys.

The module also generates a **16-byte Initialization Vector (IV)**. The IV is required by the AES-256-CBC encryption algorithm to ensure that identical plaintext encrypted with the same password results in different ciphertext.

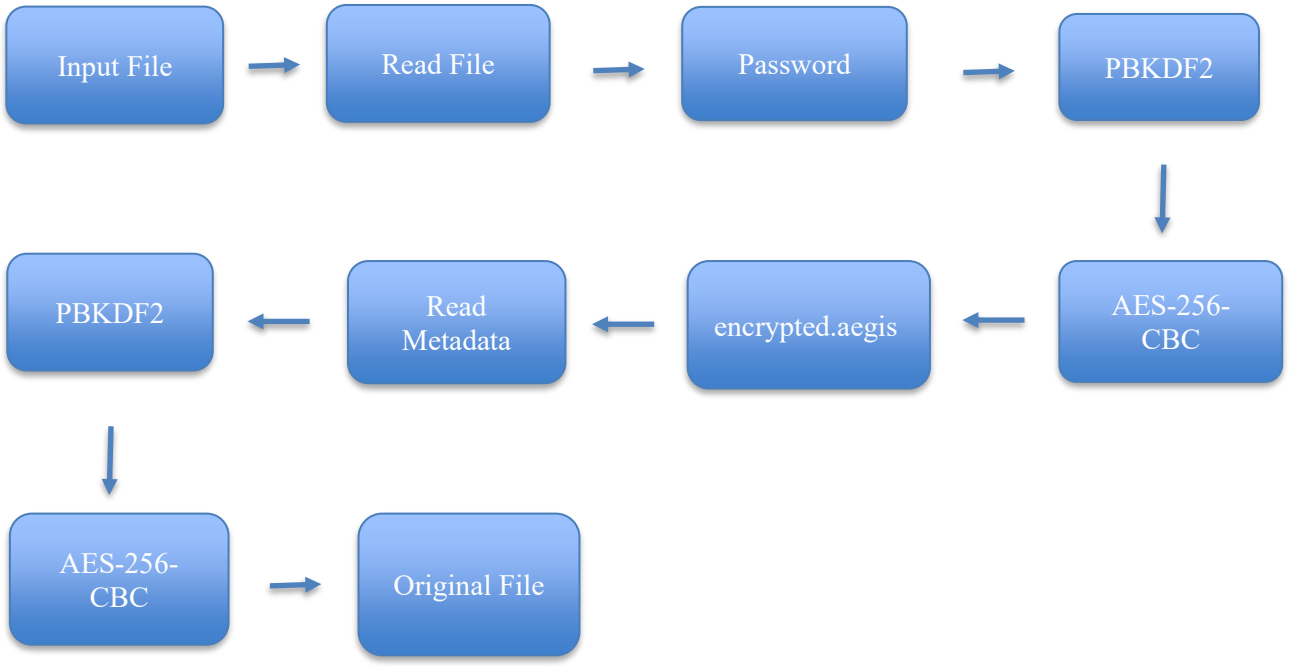
Both the salt and the IV are generated using OpenSSL's RAND_bytes() function, which provides cryptographically secure random values. These values are stored inside the .aegis file and reused during decryption to restore the original file successfully.



3.7 Data Flow of AegisVault

The data flow in AegisVault begins when the user provides an input file and password. The password is transformed into a secure encryption key through PBKDF2, while a random salt and Initialization Vector are generated using OpenSSL. The file contents are encrypted using AES-256-CBC, and the encrypted data together with the metadata is stored in the custom .aegis file.

During decryption, the encrypted file is read, the stored metadata is extracted, and the same password is used to regenerate the encryption key. The ciphertext is decrypted using the stored Initialization Vector, and the original file is restored.



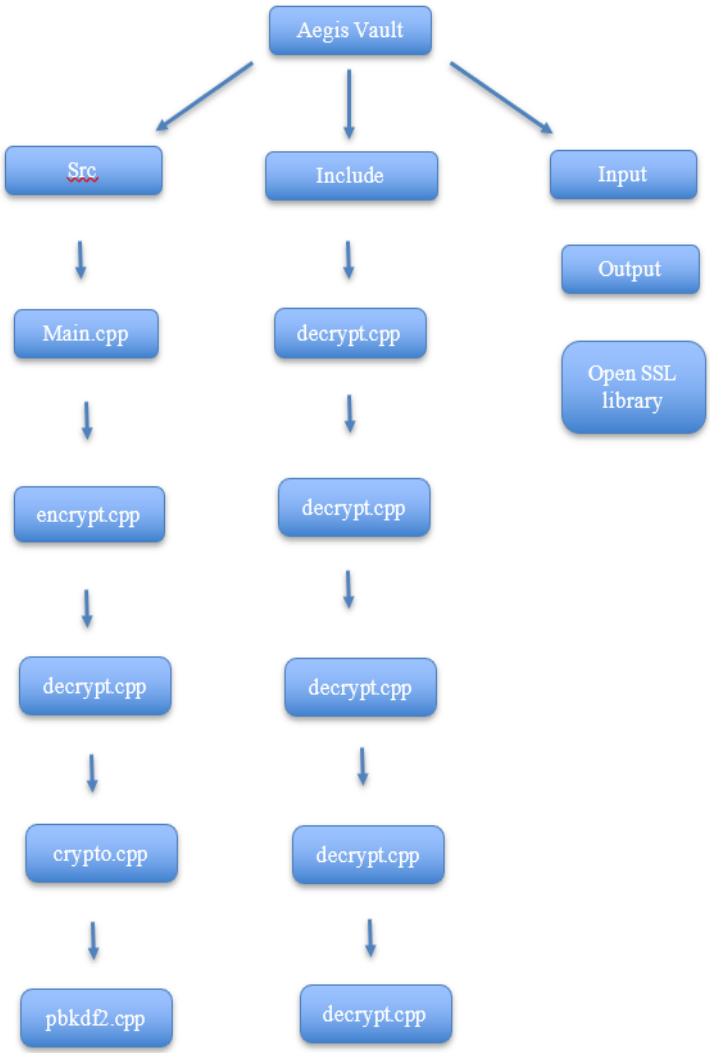
4. Project Implementation

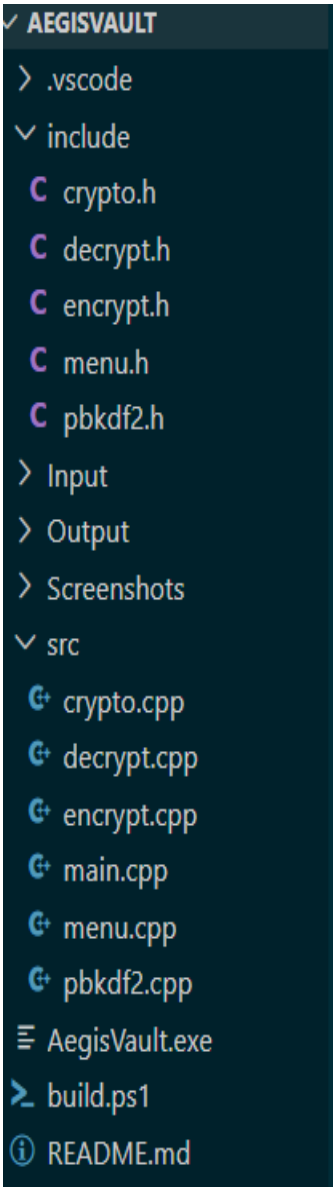
4.1 Project Structure

The AegisVault project follows a modular implementation approach in which different functionalities are separated into independent source files. Each module performs a dedicated responsibility, improving code readability, maintainability, and scalability. The project is organized using header files and implementation files, allowing different components to interact through well-defined interfaces.

The source code consists of modules responsible for user interaction, encryption, decryption, cryptographic utility functions, and password-based key derivation. The OpenSSL library is integrated into the project to perform secure cryptographic operations, while C++ standard libraries are used for file handling, memory management, and user interaction.

This structured implementation makes the project easier to understand, debug, and extend in future versions.





4.2 Main Module Implementation

The Main Module serves as the entry point of the AegisVault application. It is responsible for initiating the program, displaying the user interface, accepting user input, and directing program execution to the appropriate module.

When the application starts, the main module displays the menu containing the available operations, including file encryption, file decryption, and program termination. The user's selection is accepted through standard input and processed using a switch-case statement.

Depending on the selected option, control is transferred to either the encryption module or the decryption module. If the user selects the exit option, the application terminates gracefully after displaying a confirmation message. Invalid menu selections are handled by displaying an appropriate error message.

The main module does not perform any cryptographic operations directly. Instead, it acts as a controller that coordinates the execution of different functional modules while maintaining a simple and user-friendly interface.

```
switch(choices)
{
    case 1:
        encryptFile();
        break;
    case 2:
        decryptFile();
        break;
    case 3:
        cout<<"Thankyou for choosing AEGISVAULT encryption module..."<<endl;
        break;
    default:
        cout<<"Invalid choice..."<<endl;
```

```
------------------------------------------------------------
                AEGISVAULT encryption system
------------------------------------------------------------

1. Encrypt files
2. Decrypt files
3. To Exit

Enter your choice :
```

Figure 11: Main Menu of AegisVault

4.3 Encryption Module Implementation

The Encryption Module is implemented in encrypt.cpp and is responsible for converting the original file into an encrypted .aegis file. The module combines secure password-based key generation with AES-256-CBC encryption to provide confidentiality for stored files.

The implementation begins by accepting the file name and password from the user. The selected file is opened in binary mode and its complete contents are loaded into memory. The program then generates a random 16-byte cryptographic salt using OpenSSL and derives a secure 256-bit encryption key through the PBKDF2 algorithm.

After successful key generation, a random 16-byte Initialization Vector (IV) is generated to initialize the AES-256-CBC encryption algorithm. The OpenSSL EVP interface is then used to encrypt the plaintext file into ciphertext.

Once encryption is completed, the application creates a custom .aegis file. The file header, version number, original file extension, generated salt, Initialization Vector, and encrypted ciphertext are sequentially written into the output file. Finally, the allocated cryptographic resources are released and the encrypted file is stored in the Output directory.

This implementation ensures that sensitive information is securely encrypted while preserving the metadata required for future decryption.

```
generateSalt(salt);
derivekey(password, salt, key);

generateIV();

EVP_CIPHER_CTX* ctx = EVP_CIPHER_CTX_new();

if(ctx == nullptr)
{
cout<<"Failed to create encryption context!"<<endl;
return;
}

if(EVP_EncryptInit_ex(ctx, EVP_aes_256_cbc(), nullptr, key, iv) != 1)
{
    cout<<"Encryption initialization failed!"<<endl;
    EVP_CIPHER_CTX_free(ctx);
    return;
}
```

```
------------------------------------------------------------
                ENCRYPTION MODULE STARTED!
------------------------------------------------------------

Enter your file name with extension : sample.txt
Enter your password : sample
Reading file...

Generated Salt :
5c85e77e4c1ace93ffc9815063e33718

Generated IV(16 bytes):
ae71f8c683a29499cc1ce7861c4ce391

File encrypted successfully!
```

Figure 12: Successful File Encryption

4.4 Decryption Module Implementation

The Decryption Module is implemented in decrypt.cpp and is responsible for restoring the original file from the encrypted .aegis file. This module performs secure password verification, metadata extraction, key regeneration, and ciphertext decryption using the OpenSSL cryptographic library.

The implementation begins by requesting the user to enter the decryption password. The encrypted .aegis file is then opened in binary mode, and the application reads the stored file header, version number, original file extension, cryptographic salt, and Initialization Vector (IV).

The file header and version are verified to ensure that the selected file was created by AegisVault. If the verification fails, the application displays an error message and terminates the decryption process.

The stored salt and user password are supplied to the PBKDF2 algorithm to regenerate the same 256-bit AES key that was used during encryption. The stored IV is then used to initialize the AES-256-CBC decryption process through the OpenSSL EVP interface.

The ciphertext is decrypted block by block using EVP_DecryptUpdate(), and the remaining bytes are processed using EVP_DecryptFinal_ex(). If the password is incorrect or the encrypted file has been modified, the final decryption step fails and the application displays an **Access Denied** message without generating the output file.

If the password is correct, the decrypted data is written to the Output folder using the original file extension that was stored inside the .aegis file. Finally, all allocated cryptographic resources are released, completing the decryption process securely.

```
derivekey(password, salt, key);

EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();

if (ctx == nullptr)
{
    cout<<"Failed to create decryption context!"<<endl;
    return;
}

if (EVP_DecryptInit_ex(ctx, EVP_aes_256_cbc(), nullptr, key, iv) != 1)
{
    cout<<"Failed to initialize decryption!"<<endl;
    EVP_CIPHER_CTX_free(ctx);
    return;
}

vector<unsigned char> plaintext(ciphertext_len + EVP_MAX_BLOCK_LENGTH);
int plaintext_len = 0;

if (EVP_DecryptUpdate(ctx, plaintext.data(), &plaintext_len, ciphertext.data(), ciphertext_len) != 1)
{
    cout<<"Decryption failed!"<<endl;
    EVP_CIPHER_CTX_free(ctx);
    return;
}
int final_len = 0;

if (EVP_DecryptFinal_ex(ctx, plaintext.data() + plaintext_len, &final_len) != 1)
{
    cout<<endl;
    cout << "-----------------------------------------"<<endl;
    cout << "              ACCESS DENIED"<<endl;
    cout << "-----------------------------------------"<<endl;
    cout << "Incorrect password or corrupted encrypted file."<<endl;
    cout << "Decryption aborted."<<endl;
    EVP_CIPHER_CTX_free(ctx);
    return;
```

Figure 13 : Decryption Module

```
------------------------------------------------------------
                DECRYPTION MODULE STARTED!
------------------------------------------------------------

Decrypting file...
Enter password : image

File decrypted successfully as decrypted.jpeg
```

Figure 14: Successful File Decryption

4.5 PBKDF2 Module Implementation

The PBKDF2 Module is implemented in pbkdf2.cpp and is responsible for generating a secure 256-bit encryption key from the password entered by the user. Instead of directly using the password as the AES encryption key, the module applies the Password-Based Key Derivation Function 2 (PBKDF2) with the SHA-256 hashing algorithm.

The module combines the user password with a randomly generated 16-byte cryptographic salt and performs **100,000 iterations** to derive a secure encryption key. This significantly increases the computational effort required for brute-force and dictionary attacks.

The generated key is supplied to the AES-256-CBC encryption algorithm during encryption. During decryption, the same password and stored salt are processed through PBKDF2 to regenerate the identical encryption key. If the regenerated key matches the original encryption key, decryption proceeds successfully; otherwise, the application terminates the operation.

The implementation uses OpenSSL's PKCS5_PBKDF2_HMAC() function, ensuring compliance with modern cryptographic standards.

4.6 Cryptographic Utility Module

The Cryptographic Utility Module is implemented in crypto.cpp and provides supporting cryptographic functions used throughout the AegisVault application. The primary responsibility of this module is to generate cryptographically secure random values required during encryption.

The module generates a random 16-byte salt and a random 16-byte Initialization Vector (IV) using OpenSSL's RAND_bytes() function. The salt is supplied to the PBKDF2 algorithm for secure key derivation, while the IV is used by the AES-256-CBC encryption algorithm to ensure that identical plaintext encrypted with the same password produces different ciphertext.

By generating fresh random values for every encryption operation, the module significantly improves the overall security of the application.

4.7 Working of AegisVault

The complete working of AegisVault begins when the user selects either the encryption or decryption operation from the main menu. During encryption, the application reads the input file, accepts the password, generates a random salt and Initialization Vector, derives a secure AES key through PBKDF2, encrypts the file using AES-256-CBC, and stores the encrypted data along with the metadata inside the custom .aegis file.

During decryption, the application reads the encrypted file, verifies its header and version information, extracts the stored metadata, regenerates the encryption key using PBKDF2, and decrypts the ciphertext using the stored Initialization Vector. If the password is correct, the original file is restored with its original extension. Otherwise, the application displays an appropriate error message and terminates the decryption process.

This workflow ensures secure password-based encryption while maintaining data confidentiality and integrity.

5. Testing and results

5.1 Testing Methodology

The AegisVault application was tested to verify the correctness, reliability, and security of the implemented encryption and decryption processes. Functional testing was performed by executing different operations under normal and exceptional conditions to ensure that every module behaved as expected.

Testing included successful encryption and decryption of different file types, validation of incorrect password handling, verification of invalid encrypted files, restoration of the original file extension, and confirmation that encrypted files could not be accessed without the correct password.

Each test case was executed multiple times to verify consistency and ensure that the generated encrypted files could be successfully decrypted using the correct password.

The testing process confirmed that all implemented modules functioned correctly and satisfied the project requirements.

5.2 Test Cases

Test Case	Expected Result	Actual Result	Status
Encrypt Text File	File encrypted successfully	Successfully encrypted	Pass
Decrypt Text File	Original file restored	Successfully restored	Pass
Incorrect Password	Access Denied message	Access Denied displayed	Pass
Invalid File	Invalid file detected	Error displayed	Pass
Original Extension	File extension restored	Restored correctly	Pass

5.3 Test Result

The testing results demonstrate that AegisVault successfully performs secure password-based encryption and decryption. All functional requirements specified during the project design phase were satisfied.

The encryption module successfully generated encrypted .aegis files, while the decryption module restored the original files only when the correct password was provided. Incorrect passwords were detected correctly, preventing unauthorized access to encrypted information.

The application also preserved the original file extension after decryption and correctly handled invalid encrypted files by terminating the operation safely.
Overall, the testing confirmed that the implemented cryptographic workflow operates reliably and produces consistent results.

```
------------------------------------------------------------
                ENCRYPTION MODULE STARTED!
------------------------------------------------------------

Enter your file name with extension : artik.txt
Enter your password : artik
Reading file...

Generated Salt :
7ef2df376b404a9a80d0b57a7474643c

Generated IV(16 bytes):
70166bc2909416b7344d1259fa605cfc

File encrypted successfully!
PS C:\Users\DELL\OneDrive\Desktop\CYBER SECURITY\AegisVault>
```

Figure 15: Successful File Encryption

```
------------------------------------------------------------
                DECRYPTION MODULE STARTED!
------------------------------------------------------------

Decrypting file...
Enter password : artik

File decrypted successfully as decrypted.txt
PS C:\Users\DELL\OneDrive\Desktop\CYBER SECURITY\AegisVault>
```

Figure 16: Successful File Decryption

```
------------------------------------------------------------
                DECRYPTION MODULE STARTED!
------------------------------------------------------------

Decrypting file...
Enter password : dkah

------------------------------------------
              ACCESS DENIED
------------------------------------------
Incorrect password or corrupted encrypted file.
Decryption aborted.
PS C:\Users\DELL\OneDrive\Desktop\CYBER SECURITY\AegisVault>
```

Figure 17: Access Denied

5.4 Performance Analysis

The AegisVault application demonstrated reliable performance during testing. Encryption and decryption operations completed successfully for multiple file types without noticeable delays for normal-sized files.

The PBKDF2 algorithm with 100,000 iterations introduced a small computational delay during key generation, which is intentional and improves resistance against brute-force attacks.

Memory usage remained stable throughout execution, and no runtime errors or crashes were observed during testing. The modular implementation also simplified debugging and future maintenance of the project.

The overall performance indicates that AegisVault provides a balanced combination of security, reliability, and efficiency.

6. Conclusion

6.1 Conclusion

AegisVault was successfully developed as a secure file encryption and decryption system using **C++ and OpenSSL**. The project implements password-based key derivation using **PBKDF2 with SHA-256**, followed by **AES-256-CBC** encryption to protect the contents of user files.

The system generates a random salt and Initialization Vector (IV) during encryption and stores the required metadata along with the encrypted ciphertext in a custom `.aegis` file format. During decryption, the stored salt and IV are extracted, and the encryption key is regenerated from the user's password using PBKDF2.

The implemented system was tested under different conditions, including successful encryption and decryption, incorrect password entry, invalid encrypted files, and restoration of the original file extension. The testing results demonstrated that the application performs the intended operations successfully and handles common error conditions appropriately.

Overall, the project provided practical experience in **cryptography, secure file handling, password-based key derivation, OpenSSL, C++ programming, modular software development, and security-oriented application design**. AegisVault demonstrates how established cryptographic techniques can be combined to develop a functional file protection system.

6.2 Future Scope

Although AegisVault successfully provides password-based file encryption and decryption, several improvements can be introduced in future versions of the project.

1. **Graphical User Interface (GUI):**
   A graphical interface can be developed to make the application easier to use for users who are not familiar with command-line applications.
2. **Multiple File and Folder Encryption:**
   Future versions can support encryption and decryption of multiple files or complete folders instead of processing a single file at a time.
3. **Enhanced Authentication:**
   Additional authentication mechanisms such as multi-factor authentication or biometric verification could be integrated to provide an additional layer of security.
4. **Improved File Integrity Protection:**
   An authenticated encryption mechanism or a secure message authentication method could be incorporated to detect unauthorized modification of encrypted files.
5. **Cloud Storage Integration:**
   AegisVault could be integrated with secure cloud storage services, allowing users to encrypt files locally before securely uploading them.
6. **Cross-Platform Support:**
   The application could be extended and tested across Windows, Linux, and macOS to make it more portable.
7. **Advanced Key Management:**
   Future versions could provide a dedicated and secure key-management mechanism instead of relying entirely on user-provided passwords.

8. **Additional Encryption Algorithms:**
   Support for other modern encryption algorithms and encryption modes could be added, allowing users to select an appropriate security configuration.

REFERENCES

1. OpenSSL Project. **OpenSSL Documentation**. OpenSSL Project.
2. National Institute of Standards and Technology (NIST). **Advanced Encryption Standard (AES)**. FIPS PUB 197.
3. National Institute of Standards and Technology (NIST). **Recommendation for Password-Based Key Derivation: Part 1: Storage Applications**. NIST Special Publication 800-132.
4. National Institute of Standards and Technology (NIST). **Secure Hash Standard (SHS)**. FIPS PUB 180-4.
5. OpenSSL Project. **EVP Symmetric Encryption and Decryption Documentation**. OpenSSL Documentation.
6. C++ Reference. **Input/Output Stream Library Documentation**. Documentation for ifstream, ofstream, and file handling in C++.
7. C++ Reference. **Standard C++ Library Documentation**. Reference for C++ standard library functions and data structures used in the project.